

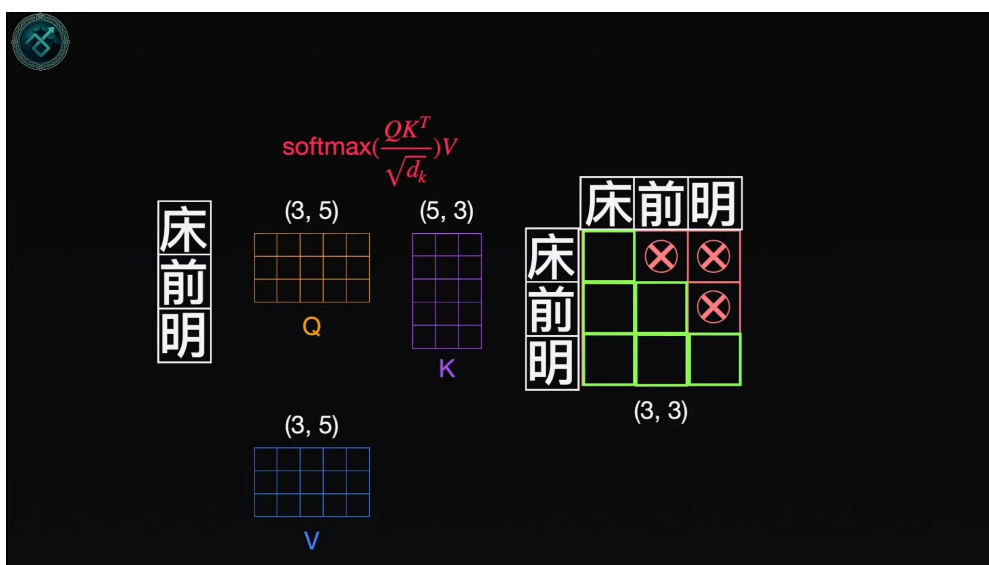
课程笔记

KV Cache：为什么它能加快大模型推理

从自回归生成的重复计算，到”算一次、存起来、下次直接读”

lecture-to-notes 自动讲义

2026 年 6 月 20 日



视频作者/频道：向量隐修会

发布平台：Bilibili

视频时长：约 13 分钟

视频链接：<https://www.bilibili.com/video/BV1SmrSBfEg6>

目录

1 引言：KV Cache 解决什么问题	2
1.1 本章小结	2
2 回顾：自注意力与因果掩码	2
2.1 从文字到 QKV	2
2.2 因果掩码：当前 token 看不到未来	3
2.3 本章小结	3
3 自回归生成：预测下一个 token 需要全部历史 K、V	3
4 KV Cache 机制：只缓存 K 和 V	3
4.1 下一轮只有”新字”需要计算	3
4.2 为什么不缓存 Q，只缓存 K、V	4
4.3 增量计算示意	5
4.4 本章小结	5
5 为什么这能大幅加速	6
5.1 本章小结	6
6 总结与延伸	6
6.1 作者的总结	6
6.2 笔者的延伸：放进整个系列看	6
6.3 拓展阅读	6

1 引言：KV Cache 解决什么问题

KV Cache 是一项能加快 *AI* 大模型输出速度的工程技术。直观地说，开启 KV Cache 后，模型可以更快地回复你。它的名字直白：**K** 和 **V** 就是自注意力机制里的 **Key** 矩阵和 **Value** 矩阵；**Cache**（缓存）的意思是——把历史上已经算过的 *K*、*V* 矩阵存起来，下次需要时直接读取，而不必重新算一遍。

要理解它为什么有效，得先回到自注意力的计算，看清楚”哪些量被反复重算了”。本讲承接上一讲《GPU 如何计算自注意力》，用一句古诗”**床前明月光**”作为例子。

1.1 本章小结

KV Cache = 缓存历史的 Key/Value 矩阵，用”空间换时间”避免重复计算。下面先回顾自注意力，再看自回归生成中的重复，最后给出缓存方案。

2 回顾：自注意力与因果掩码

2.1 从文字到 QKV

以”床前明”三个字为例。每个字是一个 **token**：先经 tokenizer 变成数字编号，再经 **嵌入**（**Embedding**）与**位置编码**变成向量，三个字堆成数据矩阵 *X*（形状 3×4 ）。再用三个参数矩阵 W_Q, W_K, W_V （各 4×5 ）与 *X* 相乘，得到 *Q, K, V* 三个矩阵（各 3×5 ）。

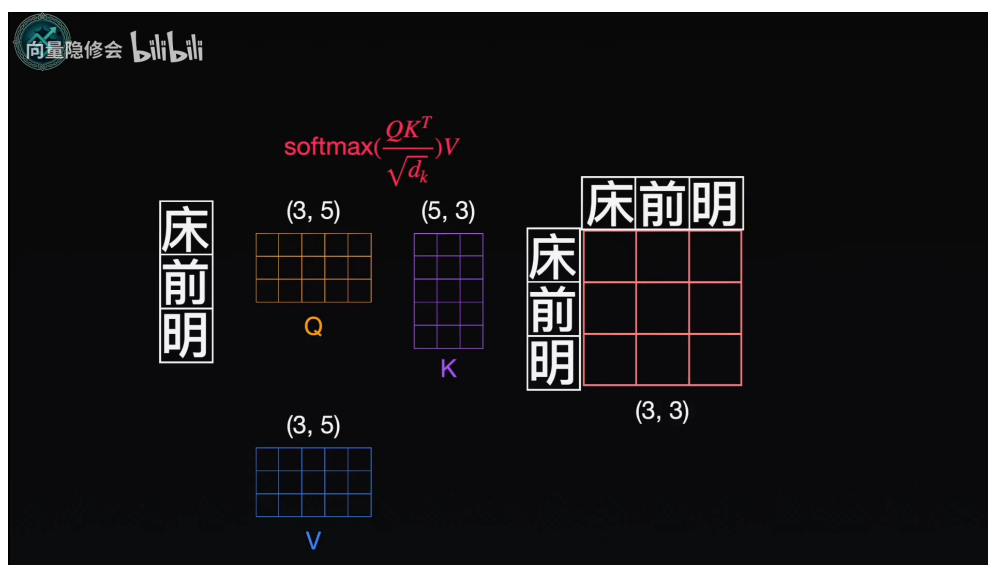


图 1：自注意力的矩阵形状： Q (3×5)、 K (5×3 转置)、 V (3×5)，经 $\text{softmax}(QK^T / \sqrt{d_k})V$ 得到 3×3 的注意力分数矩阵（行列均为”床前明”）。²

²视频画面时间区间：00:02:00–00:02:58。

自注意力的计算链（本例维度）

$$X_{3 \times 4} \xrightarrow{\times W_{Q/K/V}} Q, K, V_{3 \times 5} \xrightarrow{QK^\top} S_{3 \times 3} \xrightarrow{/\sqrt{d_k}, \text{softmax}} P_{3 \times 3} \xrightarrow{\times V} O_{3 \times 5}.$$

其中 $QK^\top$ 用 $Q (3 \times 5)$ 乘 $K^\top (5 \times 3)$ 得 3×3 ; $\sqrt{d_k}$ 是为了让训练时梯度更稳定; Softmax 逐行进行。

2.2 因果掩码：当前 token 看不到未来

GPT 类模型有一个关键特性——**因果掩码 (Causal Mask)**：当前 token 不能看到它之后的 *token*。在“床前明”里，“床”看不见“前”和“明”，只能和自己算注意力；“前”能看到“床”和自己；“明”三个都能看到。反映在 3×3 的注意力分数矩阵上，就是上三角被遮挡（屏蔽），只保留下三角。

2.3 本章小结

自注意力把输入 X 变成 Q, K, V ，再算 $\text{softmax}(QK^\top/\sqrt{d_k})V$ ；因果掩码让它成为“只能看历史”的单向注意力。这一点正是 KV Cache 能成立的前提。

3 自回归生成：预测下一个 token 需要全部历史 K、V

模型的输出是**逐字预测下一个 token**：输入“床前明”，输出“前明月”——每个位置预测的是下一个字（床 → 前，前 → 明，明 → 月）。其中“月”是模型这一步新预测出来的字。

关键观察：算出“月”这个新字，用的是 Q 矩阵的**最后一行**（即“明”对应的 query 向量），去乘完整的 $K^\top$ 、再乘完整的 V 。

预测下一个 token 需要什么

预测下一个 token 时：

- 只需要**最新一个字**的 query 向量（ Q 的最后一行）；
- 但需要**历史上所有字**的 Key 向量和 Value 向量（完整的 K 、 V ）。

这就引出了一个想法：既然每一步都要用到历史的 K 、 V ，何不**把它们存起来**？

4 KV Cache 机制：只缓存 K 和 V

4.1 下一轮只有“新字”需要计算

“月”预测出来后，下一轮输入变成“床前明月”。注意：床前明这三个字的嵌入与位置编码和上一轮完全一样，只有“月”是新的——所以新的 X 矩阵只比上一轮**多了一行**（月那一行）。

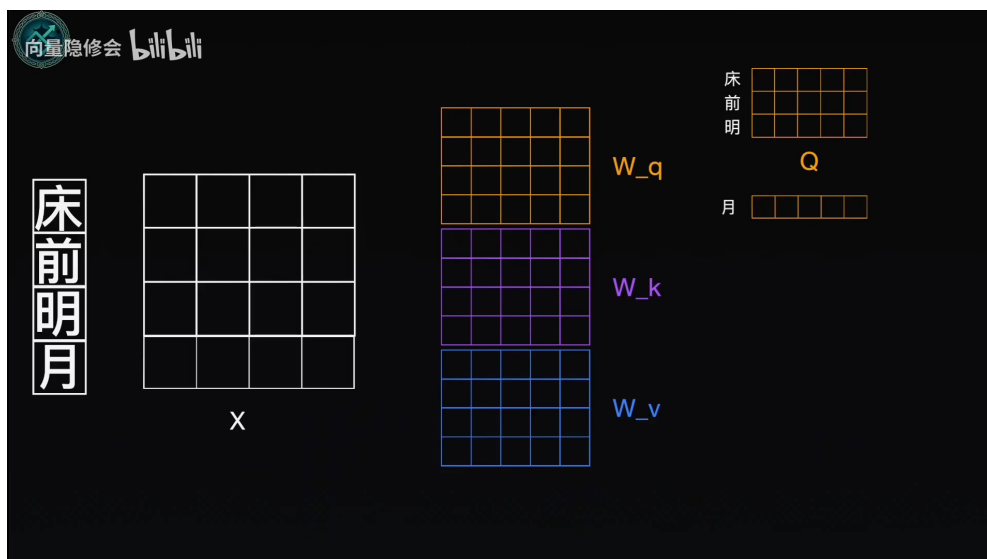


图 2: 第二轮输入”床前明月”: X 比上一轮只多了”月”这一行; 与 $W_Q/W_K/W_V$ 相乘后, Q 矩阵中”床前明”部分与之前完全相同, 只有”月”是新算的。⁴

4.2 为什么不缓存 Q , 只缓存 K 、 V

”床前明”的 Q 之前确实算过, 但——

Q 不需要缓存

我们的目的是**预测下一个 token**, 它只和最新那个字 (如”月”)的 query 向量有关, 和”床前明”这些旧字的 query **毫无关系**。所以旧的 Q 用不上, **不缓存 Q** 。而历史的 K 、 V 每一步都要用, 所以**只缓存 K 和 V** ——这正是名字”KV Cache”的由来。

于是流程变成: 历史的 K 、 V 已存好; 这一步只需算”月”对应的 k 向量、 v 向量, 把它们**堆叠到缓存的 K 、 V 矩阵后面**, 就得到完整的、当前这一步的 K 、 V 。再用”月”的 query 去和它们算注意力, 得到下一个字”光”。如此循环, 输出”前明月光”。

⁴视频画面时间区间: 00:07:24–00:08:18。

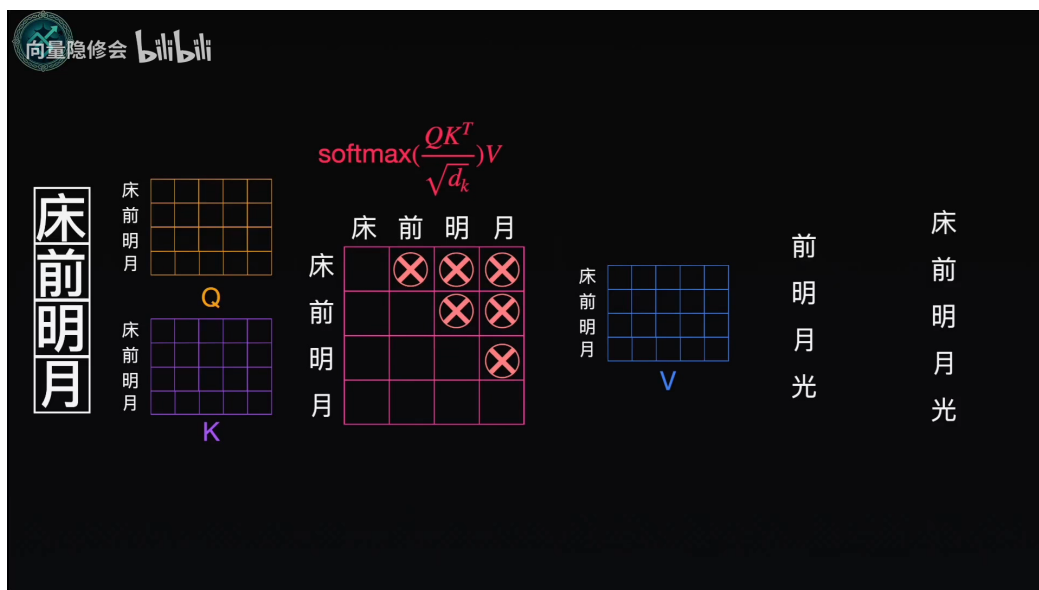


图 3: 完整一步: Q, K 都含”床前明月”四行, 注意力分数为 4×4 并施加因果掩码 (上三角被 $\otimes$ 屏蔽); 与 V 相乘后输出”前明月光”, 其中”光”是新预测的下一个字。⁶

4.3 增量计算示意

把”有无 KV Cache”的差别画出来——核心是每一步只新增一行，而不是从头重算：

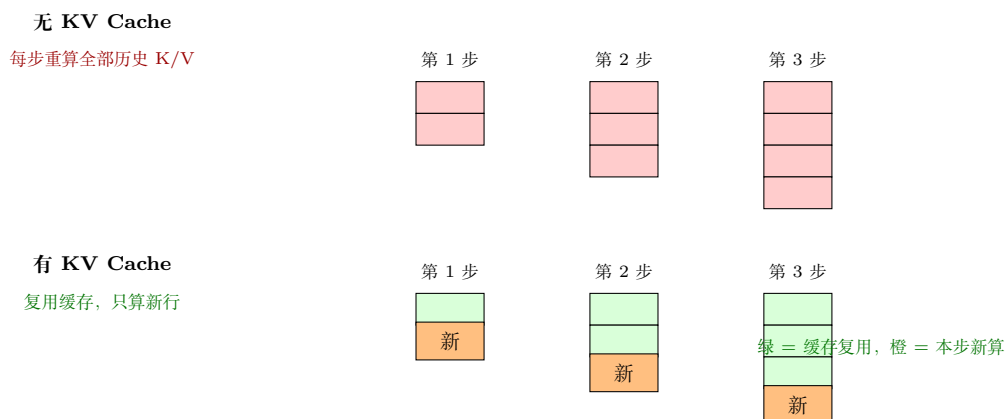


图 4: KV Cache 增量计算 (笔记自绘): 无缓存时每一步都把历史所有字的 K/V 重算一遍 (红, 计算量随长度增长); 有缓存时历史 K/V 直接复用 (绿), 每步只新算最新一个字的一行 (橙)。

4.4 本章小结

KV Cache 的全部操作就是：缓存历史 K、V；每生成一个新字，只算它的 k, v 向量并追加到缓存，而不缓存 Q 。把”每步重算整段历史”变成”每步只算新增一行”。

⁶视频画面时间区间：00:09:59–00:10:52。

5 为什么这能大幅加速

有无 KV Cache 的计算量对比

没有 KV Cache: 每生成一个字，都要把历史上所有字的 k 、 v 向量重新算一遍。若要回复用户一万个字，几乎每一步都在重复计算前面已经算过的内容——极其消耗算力和时间。

有 KV Cache: 每个字的 k 、 v 只算一次，算完存起来，下一步直接读取、不再重算。把原本随序列长度平方级增长的重复计算，压成了每步一行的增量。

5.1 本章小结

KV Cache 用显存（存 K、V）换计算，消除了自回归生成中最大的一块重复开销，是大模型推理几乎默认开启的基础优化。

6 总结与延伸

6.1 作者的总结

KV Cache 是一项加快 AI 模型推理、让用户更快看到回复的**工程技术**。原理：GPT 生成每个 token 都需要历史 token 对应的 K、V 向量才能算出下一个 token；在这个反复的过程中，把已经算过的 K、V 存起来，下一步就不必重复计算。

6.2 笔者的延伸：放进整个系列看

- **和上一讲的呼应：**上一讲《GPU 如何计算自注意力》最后点出”内存墙”——计算单元常在等 HBM 搬数据。KV Cache 从另一个角度省开销：不是省搬运，而是**省掉重复计算本身**。
- **代价是显存：**KV Cache 把时间开销转成了显存占用——序列越长、并发请求越多，KV Cache 占的显存越大。如何高效管理这块显存，正是系列下一讲 **PagedAttention** 要解决的。
- **与 FlashAttention 的分工：**FlashAttention 优化的是”单次注意力计算怎么算更省显存 IO”；KV Cache 优化的是”自回归生成怎么不重复算”。二者正交、可叠加使用。
- **一句话记忆：** Q 只看当下， KV 承载历史；历史可缓存，当下需新算。

6.3 拓展阅读

- 关键词：KV Cache、Autoregressive Decoding、Causal Mask、Prefill vs. Decode。
- 系列相关：FlashAttention（省显存 IO）、PagedAttention（KV Cache 的显存管理）。
- 视频原址：<https://www.bilibili.com/video/BV1SmrSBfEg6>（”向量隐修会”注意力优化系列）。